

Name : S.Bala Yokesh

Class : IT Batch 2

Roll no : 2025115080

DS Homework Set – 2

1) Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *ptr;
    int i;
    ptr = a;
    printf("Original array elements:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }
    for(i = 0; i < 5; i++)
    {
        *(ptr + i) = *(ptr + i) + 5;
    }
    printf("\nModified array elements:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }
    return 0;
}
```

Output :

Original array elements:

10 20 30 40 50

Modified array elements:

15 25 35 45 55

2) Write a C program to calculate the length of a string using pointers.

```
#include <stdio.h>
int main()
{
    char str[100];
    char *ptr;
    int length = 0;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    ptr = str;
```

```

while(*ptr != '\0')
{
    if(*ptr != '\n')
        length++;

    ptr++;
}
printf("Length of the string = %d", length);
return 0;
}

```

Output :

Enter a string: Hello World
Length of the string = 11

3) Write a function that calculates the sum and product of two numbers. Return these results using pointers passed as arguments to the function.

```

#include <stdio.h>
void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}
int main()
{
    int num1, num2;
    int sum, product;
    printf("Enter two numbers: ");
    scanf("%d%d", &num1, &num2);
    calculate(num1, num2, &sum, &product);
    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);
    return 0;
}

```

Output :

Enter two numbers: 10 5
Sum = 15
Product = 50

4) Write a C program to create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate, access, and modify members.

```

#include <stdio.h>
#include <stdlib.h>
struct Student
{
    int id;
    char name[30];
    float marks;
}

```

```

};
int main()
{
    struct Student *ptr;
    int n, i;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    ptr = (struct Student *)malloc(n * sizeof(struct Student));
    if(ptr == NULL)
    {
        printf("Memory allocation failed.");
        return 0;
    }
    printf("Enter student details:\n");
    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("Enter ID: ");
        scanf("%d", &(ptr + i)->id);
        printf("Enter Name: ");
        scanf("%s", (ptr + i)->name);
        printf("Enter Marks: ");
        scanf("%f", &(ptr + i)->marks);
    }
    for(i = 0; i < n; i++)
    {
        (ptr + i)->marks += 5;
    }
    printf("\nStudent Details After Modification:\n");
    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("ID   : %d\n", (ptr + i)->id);
        printf("Name : %s\n", (ptr + i)->name);
        printf("Marks : %.2f\n", (ptr + i)->marks);
    }
    free(ptr);
    return 0;
}

```

Output :

Enter the number of students: 2
Enter student details:

Student 1
Enter ID: 101
Enter Name: Arun
Enter Marks: 85

Student 2
Enter ID: 102
Enter Name: Ravi

Enter Marks: 90

Student Details After Modification:

Student 1

ID : 101

Name : Arun

Marks : 90.00

Student 2

ID : 102

Name : Ravi

Marks : 95.00

5) Define two functions with the same signature. Declare a function pointer and use the function pointer to call both functions.

```
#include <stdio.h>
void add(int a, int b)
{
    printf("Sum = %d\n", a + b);
}
void multiply(int a, int b)
{
    printf("Product = %d\n", a * b);
}
int main()
{
    void (*fp)(int, int);
    int x, y;
    printf("Enter two numbers: ");
    scanf("%d%d", &x, &y);
    fp = add;
    fp(x, y);
    fp = multiply;
    fp(x, y);
    return 0;
}
```

Output :

Enter two numbers: 10 5

Sum = 15

Product = 50

6) Declare an integer, a pointer to it, and a double pointer. Print the values and addresses at each level.

```
#include <stdio.h>
int main()
{
    int num;
    int *ptr;
    int **dptr;
```

```

printf("Enter an integer: ");
scanf("%d", &num);
ptr = &num;
dptr = &ptr;
printf("\nValue of num = %d\n", num);
printf("Address of num = %p\n", (void *)&num);
printf("\nValue of ptr (address of num) = %p\n", (void *)ptr);
printf("Value pointed by ptr = %d\n", *ptr);
printf("Address of ptr = %p\n", (void *)&ptr);
printf("\nValue of dptr (address of ptr) = %p\n", (void *)dptr);
printf("Value pointed by dptr = %p\n", (void *)&dptr);
printf("Value pointed by *dptr = %d\n", **dptr);
printf("Address of dptr = %p\n", (void *)&dptr);
return 0;
}

```

Output :

Enter an integer: 25

Value of num = 25

Address of num = 0x7ffd12345678

Value of ptr (address of num) = 0x7ffd12345678

Value pointed by ptr = 25

Address of ptr = 0x7ffd12345680

Value of dptr (address of ptr) = 0x7ffd12345680

Value pointed by dptr = 0x7ffd12345678

Value pointed by *dptr = 25

Address of dptr = 0x7ffd12345688

7) Sort an array of floating-point numbers using Selection Sort.

```

#include <stdio.h>
int main()
{
    float a[100], temp;
    int n, i, j, min;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the floating-point numbers:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%f", &a[i]);
    }
    for(i = 0; i < n - 1; i++)
    {
        min = i;
        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
            {

```

```

        min = j;
    }
}
if(min != i)
{
    temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}
}
printf("Sorted array:\n");
for(i = 0; i < n; i++)
{
    printf("%.2f ", a[i]);
}
return 0;
}

```

Output :

Enter the number of elements: 5
Enter the floating-point numbers:
3.5 1.2 4.8 2.1 0.9
Sorted array:
0.90 1.20 2.10 3.50 4.80

8) Define a structure Student with id, name, and grade. Create an array of student structures and sort them based on different fields.

```

#include <stdio.h>
#include <string.h>
struct Student
{
    int id;
    char name[30];
    char grade;
};
int main()
{
    struct Student s[100], temp;
    int n, i, j, choice;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("Enter ID: ");
        scanf("%d", &s[i].id);
        printf("Enter Name: ");
        scanf("%s", s[i].name);
        printf("Enter Grade: ");
        scanf(" %c", &s[i].grade);
    }
}

```

```

printf("\nSort By:\n");
printf("1. ID\n");
printf("2. Name\n");
printf("3. Grade\n");
printf("Enter your choice: ");
scanf("%d", &choice);
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if((choice == 1 && s[i].id > s[j].id) ||
            (choice == 2 && strcmp(s[i].name, s[j].name) > 0) ||
            (choice == 3 && s[i].grade > s[j].grade))
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}
printf("\nSorted Student Details:\n");
printf("ID\tName\tGrade\n");
for(i = 0; i < n; i++)
{
    printf("%d\t%s\t%c\n", s[i].id, s[i].name, s[i].grade);
}
return 0;
}

```

Output :

Enter the number of students: 3

Student 1

Enter ID: 103

Enter Name: Ravi

Enter Grade: B

Student 2

Enter ID: 101

Enter Name: Arun

Enter Grade: A

Student 3

Enter ID: 102

Enter Name: Kiran

Enter Grade: C

Sort By:

1. ID

2. Name

3. Grade

Enter your choice: 1

Sorted Student Details:

ID	Name	Grade
101	Arun	A
102	Kiran	C
103	Ravi	B

9) Sort a set of negative numbers using Radix Sort.

```
#include <stdio.h>
void radixSort(int a[], int n)
{
    int output[100];
    int exp, i;
    int max = a[0];
    for(i = 1; i < n; i++)
    {
        if(a[i] > max)
            max = a[i];
    }
    for(exp = 1; max / exp > 0; exp *= 10)
    {
        int count[10] = {0};
        for(i = 0; i < n; i++)
            count[(a[i] / exp) % 10]++;
        for(i = 1; i < 10; i++)
            count[i] += count[i - 1];
        for(i = n - 1; i >= 0; i--)
        {
            output[count[(a[i] / exp) % 10] - 1] = a[i];
            count[(a[i] / exp) % 10]--;
        }
        for(i = 0; i < n; i++)
            a[i] = output[i];
    }
}

int main()
{
    int a[100], temp[100];
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the negative numbers:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
        temp[i] = -a[i];
    }
    radixSort(temp, n);
    printf("Sorted array:\n");
    for(i = n - 1; i >= 0; i--)
    {
        printf("%d ", -temp[i]);
    }
}
```

```
}  
return 0;  
}
```

Output :

Enter the number of elements: 5
Enter the negative numbers:
-45 -12 -89 -23 -5
Sorted array:
-5 -12 -23 -45 -89

10) Write a C program to copy one array to another using pointers.

```
#include <stdio.h>  
int main()  
{  
    int source[100], destination[100];  
    int *p1, *p2;  
    int n, i;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
    printf("Enter the array elements:\n");  
    for(i = 0; i < n; i++)  
    {  
        scanf("%d", &source[i]);  
    }  
    p1 = source;  
    p2 = destination;  
    for(i = 0; i < n; i++)  
    {  
        *p2 = *p1;  
        p1++;  
        p2++;  
    }  
    printf("Copied array elements:\n");  
    for(i = 0; i < n; i++)  
    {  
        printf("%d ", destination[i]);  
    }  
    return 0;  
}
```

Output :

Enter the number of elements: 5
Enter the array elements:
10 20 30 40 50
Copied array elements:
10 20 30 40 50

